

3.6 The with Statement (Context Manager)

with بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 15 of 18

★ Golden

3.6 The with Statement (Context Manager) ★ Golden

LEARN IT

It is easy to forget `.close()`. If the program crashes before `.close()` runs, the file may be incomplete or corrupted. Python's `with` statement is a **context manager**: it opens the file, runs the indented block, and **always closes** the file when the block ends — even if an error occurs inside.

with open(...) as f → use f → block ends → file closed automatically

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")
# no .close() here — with already closed the file
```

After the indented block, `new_file` is closed. You do not write `new_file.close()`.

3.7 Activity: reading file content

`.read()` pulls the whole file into RAM. For a huge file that can waste memory. The professional pattern is to **iterate line by line**.

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")

with open('myfile.txt', 'r') as read_file:
    for line in read_file:
        print(line, end="")
print("\n\nFile read successfully.")
```

OUTPUT

Context managers are safe!

File read successfully.

`end=""` stops `print` from adding a second newline: each line from the file already ends with `\n`.

WORKED EXAMPLE

Three ways to read the same small file.

```

with open('sample.txt', 'w') as f:
    f.write("Ali 80\nSara 90\nAyesha 75\n")

with open('sample.txt', 'r') as f:
    print("=== .read() ===")
    print(repr(f.read()))

with open('sample.txt', 'r') as f:
    print("=== .readlines() ===")
    print(f.readlines())

with open('sample.txt', 'r') as f:
    print("=== line by line ===")
    for line in f:
        print(line, end="")

```

OUTPUT

```

=== .read() ===
'Ali 80\nSara 90\nAyesha 75\n'
=== .readlines() ===
['Ali 80\n', 'Sara 90\n', 'Ayesha 75\n']
=== line by line ===
Ali 80
Sara 90
Ayesha 75

```

`repr` shows the hidden `\n` characters. `.readlines()` is handy when you need a list (the grade tracker uses this). Line-by-line looping is best for large files because only one line sits in memory at a time.

REMEMBER IT

with = "take, use, put back." Prefer `with open(...) as f:` in every new program. You still choose the mode: `'r'`, `'w'` or `'a'`.

COMMON MISTAKE

You cannot read from `f` after the `with` block — the file is already closed. Do all reading and writing inside the block. Also, `print(line)` without `end=""` looks double-spaced because both the line and `print` add a newline.

اردو میں سمجھیں

`with` بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔ بڑی فائلیں ایک ایک سطر کے لوپ سے پڑھنا بہتر ہے بجائے پورے مواد کو `.read()` سے میموری میں بھرنے کے۔
`print(line, end="")` دگنی خالی سطر روکتا ہے۔

EXAM TIP

Define the `with` statement as a context manager that guarantees close. Contrast it with manual `open / close`. Be ready to write a four-line program that writes with `with` and a four-line program that reads line by line. Mention memory: iterating is safer for large files than `.read()`.

CHECK YOURSELF

1. What does a context manager guarantee for a file?
2. Why pass `end=""` when printing each line from a file?

Answers: 1. That the file is closed when the with block ends, even if an error occurs. 2. Because each line already ends with `\n`; print would otherwise add another newline and the output would look double-spaced.

Next lecture: 3.8 Errors and Exceptions (Try-Except)